Apresentação

MVP

MINIMUM VIABLE PRODUCT

INOVAÇÃO

Com o avanço tecnológico, a inovação passou a fazer parte do nosso dia a dia. Pensando nisso, a PUC-Rio trouxe para o aprendizado a experimentação por meio do desenvolvimento do *minimum viable product* (MVP).

Nosso propósito é preparar você para desenvolver uma solução e testá-la para atender as necessidades dos clientes, representando uma versão que é estritamente para cumprir sua função.

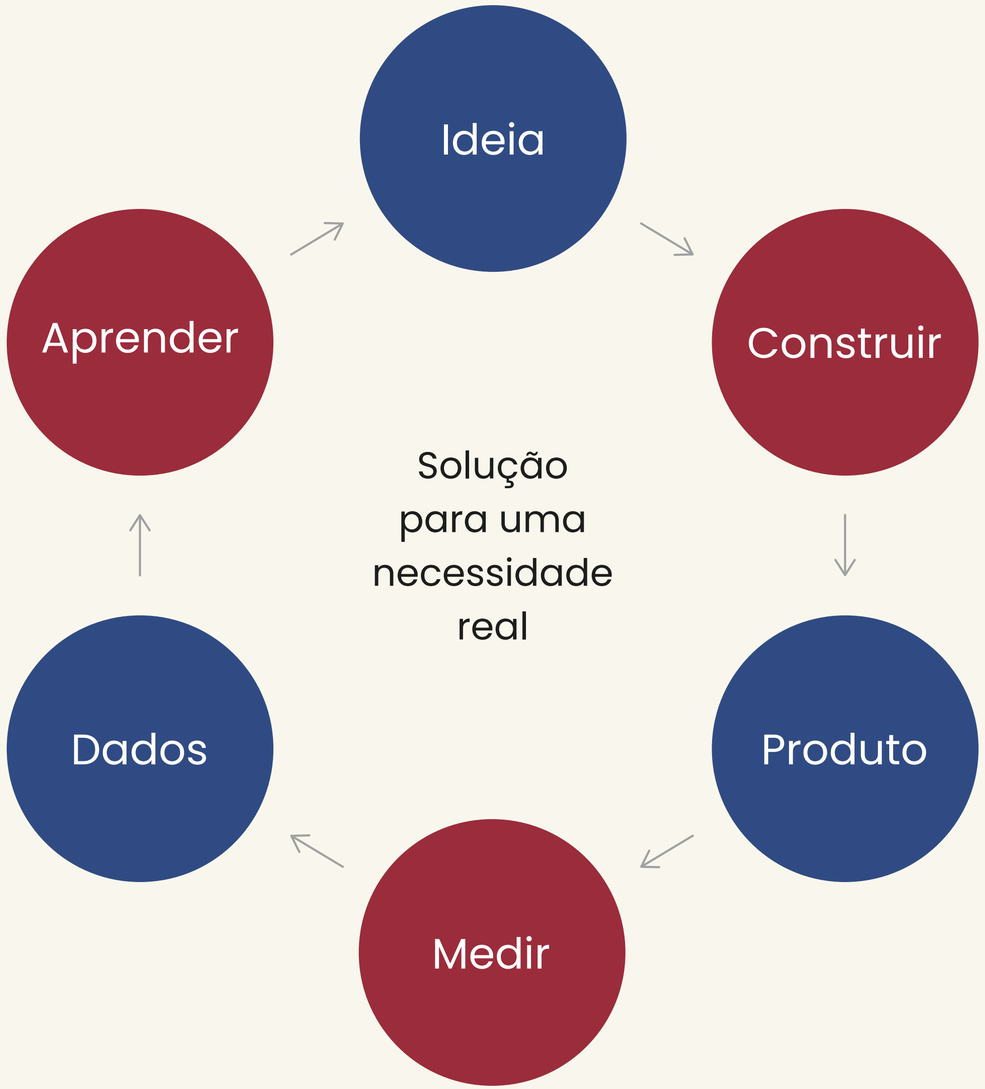
```
var val = randomInt(1, 7);
if (val === 1 ) {
    println("Signs point to yes.");
} else if (val === 2 ) {
    println("Outlook good.");
} else if (val === 3 ) {
    println("It is decidely so.");
} else if (val === 4 ) {
    println("Reply hazy, ask again.");
} else if (val === 5 ) {
    println("Concentrate and ask again.");
} else if (val === 6 ) {
    println("My sources say no.");
} else {
    println("Don't count on it.");
}
```

PROJETOS INOVADORES

BENEFÍCIOS DE DESENVOLVER UM MVP

- Testar uma ideia com poucos recursos.
- Verificar tendências de mercado.
- Lançar produtos mais rapidamente.
- Atrair investidores.

O MVP é a peça central de uma estratégia de experimentação. Para isso, você deve desenvolver uma solução real com base nos conhecimentos adquiridos em sua *sprint*.



Etapas para o desenvolvimento do MVP

Ideia

Qual será a principal funcionalidade do MVP?

Estrutura

Que ferramentas serão utilizadas para o desenvolvimento?

Protótipo

Entrega do protótipo final

MUITO BEM!

Agora que você conheceu um pouco mais sobre o MVP, está pronto para iniciar o desenvolvimento do seu.

Fique atento às etapas e aos prazos de entrega.

Requisitos para o Desenvolvimento do MVP

Contexto:

1. Escolha um *dataset* que possa ser utilizado em um problema de classificação que não tenha sido utilizado na disciplina de *Engenharia de Software para Sistemas Inteligentes*. Exemplos de onde você pode encontrar *datasets* interessantes:

- UCI Machine Learning Repository: https://archive.ics.uci.edu/datasets
- Kaggle: https://www.kaggle.com/datasets
- Google Datasets: https://datasetsearch.research.google.com/
- Hugging Face: https://huggingface.co/datasets

Aproveite os filtros que os repositórios oferecem para encontrar com mais facilidade o *dataset* da sua preferência. Caso você prefira usar um *dataset* que reflita um problema real da sua empresa (cuidado apenas com a confidencialidade dos dados), será muito bem-vindo.

2. Você deverá treinar um modelo de *machine learning* utilizando métodos clássicos para um problema de classificação. Deverão ser contempladas as etapas estudadas na disciplina de *Engenharia de Sistemas de Software Inteligentes:* carga dos dados, incluindo a separação entre treino e teste (*holdout*), transformação de dados (normalização e padronização), modelagem (utilização dos algoritmos KNN, Árvore de Classificação, Naive Bayes e SVM), otimização de hiperparâmetros, avaliação e comparação de resultados dos modelos treinados com os diferentes algoritmos e exportação do modelo resultante. Recomenda-se o uso de *cross-validation* e a criação de *pipelines*, conforme apresentado no material do curso.

3. Produza um *notebook* no Google Colab (considerando o item 1 e o item 2) com as características a seguir:

a. O *notebook* servirá como relatório da resolução do problema de *machine learning*, descrevendo textualmente (utilizando as células de texto) o contexto do problema e cada uma das operações realizadas;
b. É obrigatória a utilização da linguagem Python e da biblioteca *Scikit-Learn*. Bibliotecas adicionais podem ser utilizadas, caso necessário.

c. Crie o *notebook* seguindo as boas práticas de codificação vistas na disciplina *Programação Orientada a Objetos*, mas para esta parte não é necessário que você crie classes no seu código.

4. Desenvolva uma aplicação *full stack* (contemplando *back-end* e *front-end*) simples, utilizando as tecnologias estudadas na sprint *Desenvolvimento Full Stack Básico*. Esta aplicação deverá somente fazer a carga do arquivo do modelo de *machine learning* no *back-end* (o modelo será servido de forma embarcada no *back-end*), possibilitar a entrada de novos dados no *front-end* para que o modelo de classificação faça a predição da classe de saída e exibir o resultado na tela.

5. Implemente um teste automatizado utilizando PyTest para assegurar que o modelo atenda aos requisitos de desempenho estabelecidos (por você). Para testar as predições do modelo é preciso escolher métricas adequadas e thresholds (valores limite) aceitáveis para o desempenho do modelo, para determinar se o teste passou ou falhou. Este teste deverá permitir, no caso da substituição do modelo, evitar a implantação de um modelo que não atenda aos requisitos de desempenho.

6. Reflita sobre como as boas práticas vistas na disciplina *Desenvolvimento de Software Seguro* (por exemplo, técnicas de anonimização de dados) poderiam ser aplicadas ao seu problema.

Observações:

- O *dataset* pode ser selecionado de acordo com a sua escolha, desde que seja diferente dos *datasets* vistos na disciplina de *Engenharia de Sistemas de Software Inteligentes.* O *dataset* deve ser carregado através de uma URL dentro do próprio *notebook*, de forma a permitir a execução direta do código sem necessidade de realizar qualquer tipo de configuração ou ajuste. Veja um exemplo de importação através de URL no link https://colab.research.google.com/github/dipucriodigital/ciencia-de-dados-e-analytics/blob/main/analise-exploratoria-pre-processamento-de-dados/AEPD_importando_datasets.ipynb (forma 1 do *notebook*).

- Exemplo de como ler o *dataset* a partir do seu repositório do GitHub: colocar o arquivo em um repositório seu do GitHub e referenciá-lo na URL com a sua versão *raw*, por exemplo:

https://raw.githubusercontent.com/tatianaesc/datascience/main/diabetes.csv

- Apesar de recomendarmos a utilização do Colab para a etapa da construção do modelo de *machine learning*, você pode construir o seu código na IDE que desejar. Porém, a entrega final para esta parte do MVP deverá *obrigatoriamente* ser um *notebook* aberto no Colab.

Sobre a entrega:

Na resolução da tarefa do MVP, no ambiente do curso, você deverá informar **o link do seu repositório público no GitHub**, que deverá conter:

- **O *notebook*** (em formato Colab) contendo o processo de criação do modelo de *machine learning* (conforme detalhado nos itens 1 a 3 da seção de contexto). Este *notebook* deve poder ser executado de forma independente da aplicação *full stack*, sem erros, do início ao fim, usando o Google Colab (no menu *Arquivo* no Google Colab, selecionar a opção *Salvar uma cópia no GitHub*);
- **O código completo da aplicação *full stack* desenvolvida**, incluindo o teste do modelo (itens 4 e 5);
- **Um vídeo** de no máximo 3 minutos mostrando a aplicação rodando e chamando o modelo de *machine learning*;
- Eventuais arquivos auxiliares necessários para a execução do seu código;
- **IMPORTANTE: Garanta que o link informado funcione quando fizer a entrega do seu MVP!**

Requisitos e composição da nota:

NOTEBOOK DE MACHINE LEARNING:

- ***(1,0 pt)* Execução sem erros:** o *notebook* deve poder ser executado pelo professor, do início ao fim, sem erros, no ambiente Google Colab.
- ***(2,0 pt)* Processo de criação do modelo e documentação consistente:** siga corretamente as etapas descritas no item 2 (da seção de contexto) e utilize blocos de texto que expliquem cada etapa e cada decisão do seu código, contando uma história completa, consistente e compreensível, do início ao fim.
- ***(1,0 pt)* Análise de resultados do modelo:** no final do *notebook*, você deverá escrever um bloco de texto resumindo os principais achados, analisando os resultados e levantando eventuais pontos de atenção. Sugerimos também incluir um bloco de texto de conclusão do problema

como um todo, resumindo os principais pontos e fazendo um fechamento desta parte do trabalho.

APLICAÇÃO FULL STACK:

- ***(2,0 pt)* Integração do modelo de *machine learning* com a aplicação *full stack*:** a aplicação deverá fazer a carga do arquivo do modelo de *machine learning* no *back-end*, possibilitar a entrada de novos dados no *front-end* e exibir o resultado da predição na tela.
- ***(1,0 pt)* Teste automatizado do modelo no *back-end* da aplicação:** o teste deverá assegurar que o modelo atenda aos requisitos de desempenho estabelecidos.
- ***(1,0 pts)* Código limpo, capricho e qualidade dos códigos:** seu código deve estar legível e organizado, utilizando as boas práticas de codificação em Python.

VÍDEO:

- ***(2,0 pts)*** Qualidade, completude, objetividade e clareza do vídeo de apresentação do trabalho.

- ***03/03:* Orientação inicial do MVP**
- ***17/03:* Encontro 1 de dúvidas do MVP**
- ***24/03:* Encontro 2 de dúvidas do MVP**
- ***31/03:* Encontro 3 de dúvidas do MVP**
- ***07/04:* Encontro 4 de dúvidas do MVP**
- ***12/04:* Entrega do MVP**
- ***20/04:* Lançamento de notas do MVP**